



Figure 4: Access Control List has been created for the client

Next, create an IQN for the target. This step will also create a default TPG underneath the IQN.

An administrator can use `create` without specifying the IQN to create. `targetcli` will generate an IQN similar to the following:

```
iqn.2018-01.com.example:server.
```

Specifying the IQN value provides the administrator the ability to use a meaningful namespace for their IQNs.

Next, in the TPG, create an ACL for the client node to be used later. Because the global `auto_add_mapped_luns` parameter is set to true (default), any existing LUNs in the TPG are mapped to each ACL as it is created.

The ACL configures the target to only accept initiator connections from a client presenting `iqn.2018-01.com.example:client` as its initiator IQN, also known as the initiator name.

Next, in the TPG, create a LUN for each existing backstore. This step also activates each backstore. Because ACLs exist for the TPG, they will automatically be assigned to each LUN created.

A number of LUNs assigned to a target means that when the initiator connects to the target, it will receive *n* number of SCSI devices.

Now, still inside the TPG, create a portal configuration to designate the listening IP address and ports. Create a portal using the system's public networks interface. Without specifying the TCP ports to be used, the portal creation will default to the standard iSCSI port 3260.

Next, view the entire configuration, then exit `targetcli`, which will automatically save upon exit. The resulting persistent configuration file is stored in JavaScript Object Notation (JSON) format.

Now add a port exemption to the default firewall for port 3260, the standard iSCSI port, as follows:

```
# firewall-cmd --add-port=3260/tcp
# firewall-cmd --add-port=3260/tcp --permanent.
```

In the final step, enable the `target.service systemd` unit. The `target.service` will recreate the target configuration from the `json` file at boot. If this step is skipped, any configuration target will work until the machine is rebooted. However, after a reboot, no target will be offered by the server.

```
# systemctl enable target.
```

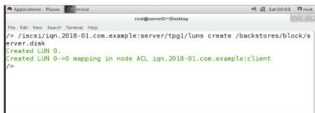


Figure 5: LUN created for backstore



Figure 6: Portal has been created for listening IP address and port

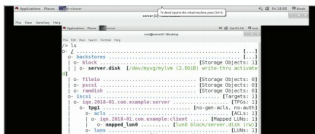


Figure 7: Final configuration

Authentication

In addition to ACL node verification, password based authentication can be implemented. Authentication may be required during the iSCSI discovery phase, and it can be unidirectional or bi-directional.

CHAP authentication does not use strong encryption for the passing of credentials. While CHAP does offer an additional factor of authentication besides having a correctly configured initiator name configured in an ACL, it should not be considered as secure enough, if security of iSCSI data is of concern. Controlling the network side of the protocol is a better method to ensure security. Providing a dedicated, isolated network or VLANs to pass the iSCSI traffic will be a more secure implementation of the protocol.

An introduction to iSCSI initiators

In Red Hat Enterprise Linux, an iSCSI initiator is typically implemented in software and functions similar to a hardware iSCSI HBA to access targets from a remote storage server. Using software based iSCSI initiators requires connecting to an existing Ethernet network of sufficient bandwidth to carry the expected storage traffic.

iSCSI can also be implemented using a hardware initiator that includes the required protocols in a dedicated host bus adaptor. iSCSI HBAs and TCP offload engines, which includes the TCP network stack on an Ethernet NIC, and move the processing of iSCSI or TCP overhead and Ethernet interrupts to hardware, easing the load on system CPUs.